SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

GROUP : YOLOO!!

Members:

1) **Mohammad Afiq Zakwan Bin Asri**
2) **Akmal Hariz Bin Azidi**
3) **Adib Rusyaidi Mohd Zaki**
4) **Nur Khadijah Aleya Binti Azizul**
5) **Nur Dianah Binti Amir Chong**

1. Introduction

This System Analysis Document explains how SmarTax works behind the scenes. While the Product Requirement Document focuses on what SmarTax does and why, this document dives into how the system releases those requirements. Outlining the system's technical architecture, data flows, and key design decisions that shape the platform, where OpenAI integration sits in the request path, how data moves through the system, and the trade-offs have been made consciously. In short, this document is about turning the product vision into a working, real-world system.

1.1. Purpose:

This SAD serves three primary audiences:

1. Developers on the YOLO!! Team, who need a reference for how the codebase is organised and where to make changes.
2. QA engineers, who need to understand system boundaries in order to design meaningful integration and regression tests.
3. Hackathon judges, who need a concise, credible demonstration that SmarTax is architecturally sound and not merely a UI wrapper over an API call.

Key Elements Covered in This Document

- **Architecture**
  SmarTax is a single-repository full-stack web application deployed on Vercel with Supabase as the managed backend. A single Next.js codebase exposes both the React frontend and the Node.js API routes, with OpenAI integrated as a first-class service layer rather than an opaque third-party call.

- **Data Flows**
  User inputs (EA forms, P&L statements, and wizard answers) move through a deterministic extraction, retrieval, and reasoning pipeline, with GLM consulted only for tasks it is uniquely qualified to perform.

- **Model Process**
  End-to-end workflow for the three core features — relief finding, tax calculation, and pre-year-end action planning — is documented as a sequence of system interactions.

- **Role of Reference**
  This document is the single source of truth for the high-level architecture and is the starting point for onboarding new contributors and for preparing the technical walkthrough during the pitch.

1.2. Background:

Malaysia's current tax-filing landscape is split between LHDN's e-Filing portal (which is a submission tool with no decision support) and professional tax agents (which are costly and inaccessible to most filers). Several commercial "tax calculators" exist online, but they are purely numerical and offer no personalised relief discovery, no grounded reasoning, and no pre-year-end planning. SmarTax is the first system of its kind in Malaysia to combine structured retrieval over LHDN Public Rulings with an LLM reasoning layer on top of a deterministic tax engine.

1.2.1. Previous Version

No prior version of any AI-Driven Tax Filing Automation System exists. Rather than an existing system, it exists as a fragmented workflow.

- **LHDN e-Filing portal** – functions purely as a submission interface. It validates inputs and computes tax based on declared values but provides no advisory, optimisation, or proactive guidance.
- **Manual tax planning** – taxpayers rely on personal knowledge, ad hoc research, or informal advice. This leads to underutilisation of eligible reliefs due to lack of awareness or misunderstanding of eligibility rules.
- **Professional tax agents** – offer advisory and optimisation, but are cost-prohibitive and not scalable for the general population.
- **Online tax calculators** – deterministic tools that compute tax payable but lack contextual reasoning, personalised recommendations, and integration with authoritative sources (e.g., LHDN Public Rulings).

Overall, the "previous version" is a manual, rule-based ecosystem with no intelligent, integrated decision-support layer.

1.2.2. Changes in Major Architectural Components

Since no prior version of SmarTax exists, these changes describe what SmarTax introduces compared to the current fragmented landscape as listed above.

Current Fragmented Workflow	What SmarTax Changes
No system tells users which reliefs they qualify for	SmarTax searches LHDN rulings and matches reliefs to the user's situation automatically.
Tax calculators only show final numbers with no explanation	SmarTax shows a full step-by-step breakdown from gross income to final tax, with plain-language reasoning behind every relief.
No tool cites the actual LHDN law behind its recommendations	SmarTax links every recommendation to the exact Public Ruling, ITA section, and paragraph.
No system helps users plan before year-end	SmarTax generates a pre-year-end action plan with deadlines and estimated savings.
AI chatbots (ChatGPT etc.) answer tax questions but make things up	SmarTax answers questions using only retrieved LHDN rulings (RAG), with a citation firewall that blocks made-up answers.

1.2.3. New Capabilities Introduced

- **Intelligent relief discovery with RAG grounding.** SmarTax implements intelligent tax relief discovery within the chatbot by using RAG-based retrieval over LHDN Public Ruling documents stored in a vector database (pgvector).
- **Citation-backed trust system.** Every relief and chat answer comes with a clickable citation badge showing the exact LHDN Public Ruling, ITA section, and paragraph. A firewall layer checks every citation against the database before showing it to the user. Made-up rulings are blocked automatically.
- **Separated AI and calculation layers.** GLM handles understanding (reading documents, finding reliefs, explaining in plain language). A separate TypeScript engine handles the math (tax bands, relief caps, rebates). The two never mix. This means the numbers are always accurate and auditable.
- **Natural-language tax Q&A.** Users can ask any tax question in their own words (e.g. "Can I claim my MacBook?"). SmarTax retrieves the relevant LHDN ruling chunks from the database and gives an answer grounded in real law, not AI memory.
- **Multi-mode architecture.** One codebase serves Individual, SME, and Freelancer users. Each mode loads different relief databases, different tax band logic, and different GLM prompt templates. Users pick their mode on the landing page.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Individual Filer (Aminah)	Employee filing annual income tax. Uploads EA form, completes wizard, acts on recommendations.	Simple, trustworthy flow with clear RM figures; bilingual support; no jargon; citations behind every claim.
SME Owner (Encik Razak)	Owner-operator of a small company. Uploads P&L, reviews deductible-expense categorisation.	Identifies missed capital allowances and deductions; explanations at accountant-quality depth; mode-appropriate UI.
Freelancer (Jia Wen)	Mixed-income filer. Uses chat interface heavily.	Natural-language answers to ad-hoc questions; clarity on Form B vs Form BE; handling of business expenses.
Development Team	Builds, deploys, and maintains SmarTax.	Clean API contracts; modular code; clear prompt templates; CI checks prevent breaking changes.
QA Team	Validates system behaviour across all three modes.	Defined input/output schemas; explicit edge-case coverage; documented retry and fallback behaviour.
Hackathon Judges	Evaluate the product against the UMHackathon 2026 rubric.	Evidence that GLM is load-bearing; technical walkthrough showing sound architecture; demonstrable impact.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

SmarTax is structured as a single-repository full-stack Next.js 14 application deployed on Vercel, with Supabase providing managed Postgres (including pgvector), authentication, and file storage as its backend. Z.AI GLM is accessed through a dedicated service-layer module that enforces prompt templates, schema validation, caching, and fallback behaviour before any request reaches the external API. The same Next.js codebase serves three user modes — Individual, SME, and Freelancer — differentiated at runtime by routing and prompt templates rather than by separate applications.

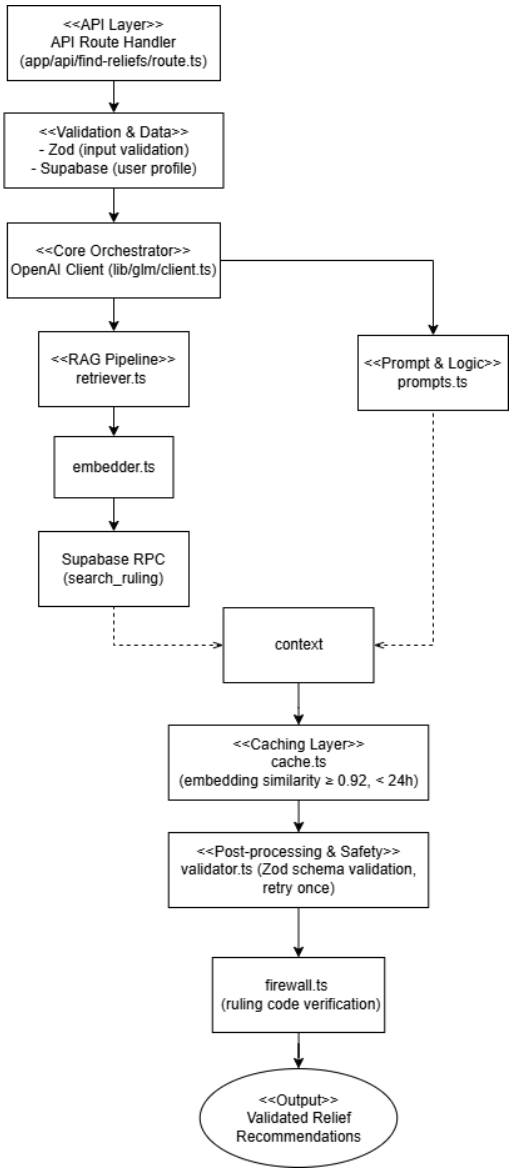
The architecture is deliberately organised in four horizontal tiers:

- **Presentation tier.** React 18 components rendered via the Next.js App Router. Styling is handled through Tailwind CSS with shadcn/ui components. All state is client-side or in URL search parameters; no global store is needed for the MVP.
- **API tier.** Next.js Route Handlers exposing six endpoints: /api/extract-ea, /api/extract-pnl, /api/find-reliefs, /api/calculate-tax, /api/action-plan, and /api/ask. Each handler is thin: it validates input with Zod, delegates to a domain module, and returns a typed JSON response.
- **Domain tier.** Four pure modules that contain the business logic: lib/tax/engine.ts (deterministic calculation), lib/glm/ (prompt templates and API wrapper), lib/rag/ (embedding and retrieval), and lib/pdf/ (PDF parsing). These modules have no knowledge of HTTP; they are importable and unit-testable in isolation.
- **Data tier.** Supabase Postgres with seven tables (users, filings, reliefs_master, filing_reliefs, rulings_vectors, action_plans, chat_history). Vector search is implemented as a SQL function search_rulings that uses pgvector's cosine-similarity operator.

2.1.2 LLM as Service Layer

A common mistake in AI products is to treat the language model as an opaque "AI Module" that receives a request and returns a response. SmarTax deliberately treats GLM as a service layer with explicit contracts, validation, caching, and fallback behaviour — the same discipline applied to any other external dependency. This is implemented in the lib/glm/ module and is best understood as a dependency diagram.

Dependency Diagram



Prompt Construction and Context Assembly

Every **OpenAI API call** is constructed using four structured input components, maintained in a strict order to ensure consistent and grounded responses:

1. **System prompt (static, per endpoint).** Defines the model's role as a Malaysian tax advisory assistant. Restricts fabrication of tax rules or references, and specifies the expected output structure for each task.

2. **Retrieved ruling chunks (dynamic, from pgvector).** Top-5 passages ranked by cosine similarity against the query embedding, each chunk capped at 500 tokens.

3. **User profile (dynamic, from Supabase).** A structured JSON object derived from user-filled inputs, including business income, expenses, selected options from the wizard, and any declared reliefs.

4. **Task instruction (static, per endpoint).** The specific request for this endpoint: "extract this document", "find eligible reliefs", "generate an action plan", or "answer this question".

Token Limit Enforcement

Token budgets are enforced at two checkpoints: (1) input-side, where each slot is capped before concatenation — system prompt ~400 tokens, retrieved rulings ~2,500 tokens, user profile ~800 tokens, instruction ~200 tokens — yielding a total input of approximately 4,000 tokens; and (2) output-side, where max_tokens is set per endpoint (1,500 for relief finding, 600 for action plan items, 500 for chat answers).

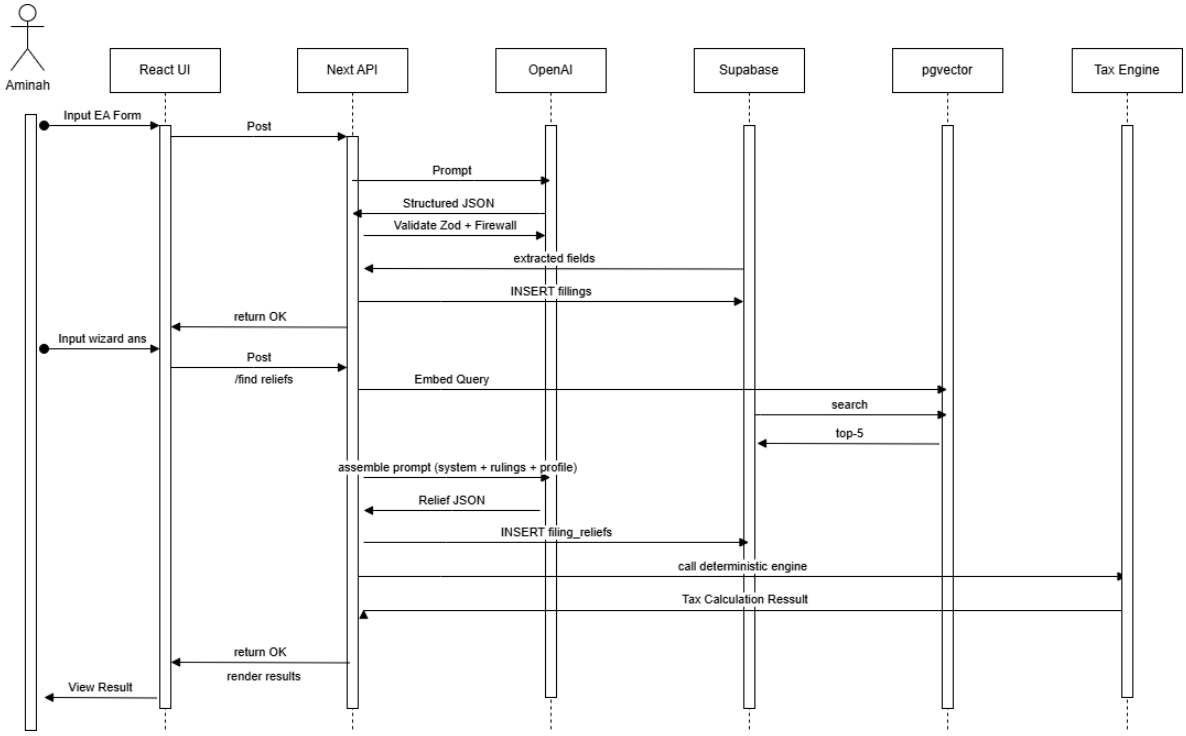
Response Parsing

All responses are parsed by endpoint-specific Zod schemas defined in lib/glm/schemas.ts. A parse failure triggers a single retry with an explicit repair prompt. A second failure returns a typed error to the API handler, which responds to the client with a graceful error state. Successful parses flow downstream to the Tax Engine or directly to the API response.

Major API Calls Between System Components

From	To	Purpose / Payload
Next.js /api/find-reliefs	Supabase RPC (search_rulings)	Vector-similarity search over rulings_vectors. Returns top-5 ruling chunks.
Next.js /api/calculate-tax	lib/tax/engine.ts	Pure in-process function call. No external I/O. Returns TaxCalculationResult.
Next.js /api/calculate-tax	Supabase filings table	INSERT/UPDATE with final numbers for persistence.
Next.js /api/ask	Supabase RPC	Vector-similarity search for the user's question.
Next.js /api/ask	Z.AI GLM API	Chat prompt with question + retrieved rulings + optional user context. Returns cited answer.
React UI	Supabase (direct)	Authentication (sign-in, sign-up, session refresh) uses Supabase's client SDK directly from the browser.
React UI	Supabase Storage	Signed URL uploads for EA form and P&L files; signed URL reads for document viewing.

2.2.2. Sequence Diagram



Each of the core features—relief identification, tax calculation, and action planning—follows a consistent processing pattern aligned with the sequence flow. Instead of document extraction, all inputs are now collected directly through structured form inputs and chatbot interactions.

The flow begins with **input validation**, ensuring user-provided data from the UI is complete and well-formed. Next, the system performs **context retrieval** by querying relevant LHDN rulings from the vector database using similarity search. This retrieved context, together with the user's profile data, is used to **construct a prompt for the OpenAI model**.

The model then generates a structured response (e.g., eligible reliefs or recommendations), which is subsequently passed through a **validation and firewall layer** to prevent invalid or non-compliant outputs. The validated results are then **persisted into Supabase** for tracking and reuse, before being returned to the UI for display.

This standardized pipeline—input validation → context retrieval → OpenAI call → response validation → persistence → UI response—ensures consistency across features, making the system easier to maintain, test, and extend.

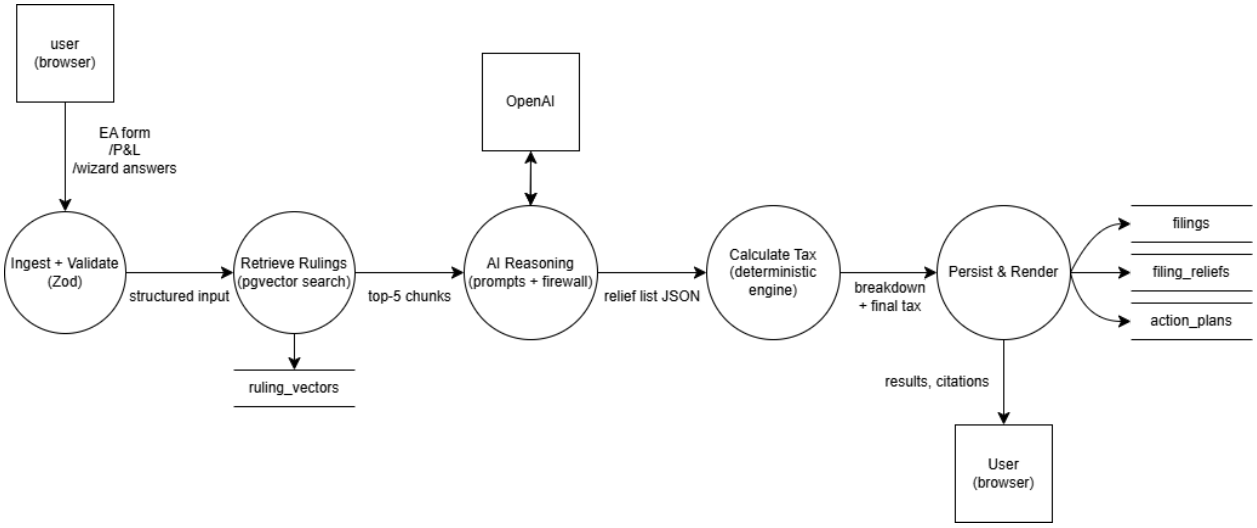
2.2. Technological Stack

Frontend	Next.js 14 (App Router) + React 18 — Single-codebase full-stack framework. Eliminates separate frontend/backend repos and cross-origin issues. Supports both static pages and serverless API routes in one project.
Backend	Next.js API Route Handlers (Node.js runtime) — No separate server to deploy or debug. Share type definitions with the frontend. Deploys to Vercel serverless functions automatically.
Language	TypeScript — Enforces typed contracts between fronted, API, domain modules , and database. Particularly valuable for GLM response validation via Zof inference.
Database	Supabase PostgreSQL 15 — Managed Postgres with zero setup. Row-level security enforces per-user data isolation. Automatic daily backups on the free tier.
Vector Search	pgvector — Keeps vector search in the same database as the relational data. Removes the need for a separate vector DB like Pinecone.
Authentication	Supabase Auth — Email/password, Google, and GitHub sign-in out of the box. JWT-based sessions integrate cleanly with Next.js middleware.
File Storage	Supabase Storage - Bucket-level access policies.
AI Reasoning	OpenAI — Used to understand user input and generate tax advice. Responses are supported by relevant LHDN rulings retrieved via vector search to ensure accuracy.
AI Embedding	OpenAI text-embedding-3-small — Generate vector embeddings for semantic similarity search.
Validation	Zod — Runtime schema validation for API inputs and for GLM outputs. Generates TypeScript types from the same schema.
Deployment	Vercel (hobby tier) — One-command deploy from GitHub. Automatic HTTPS, edge caching, preview deployments for every pull request.
CI/CD	GitHub Actions — Runs TypeScript type-check, ESLint, Vitest unit tests, and a staging deploy on every push to main.
Monitoring	Vercel Analytics + Supabase Logs — Free-tier observability sufficient for the MVP. Vercel provides traffic and web-vitals metrics; Supabase exposes query logs and auth events.

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

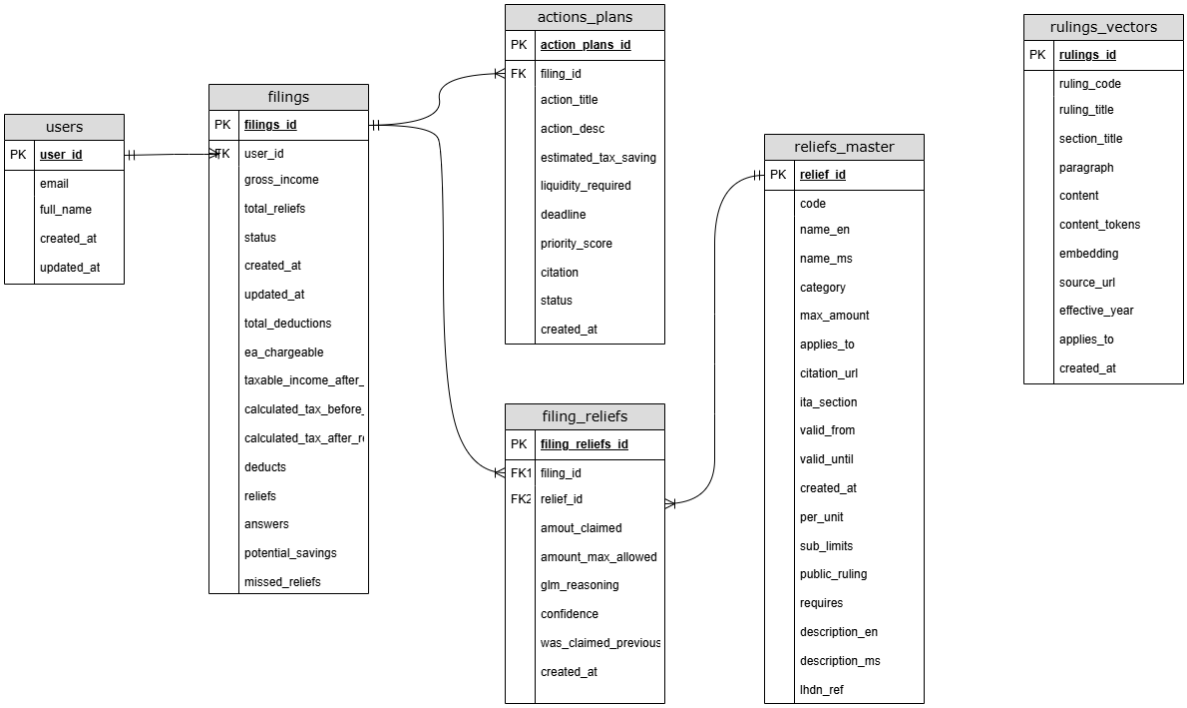
The DFD below traces the complete path of data from the user's browser through the SmarTax pipeline and back. External entities are Users and Z.AI GLM; data stores are Supabase Postgres tables; processes are the API handlers and domain modules. No black-hole stores are present — every data store has at least one read and one write flow.



Every data store has bidirectional flow:

- **rulings_vectors** — WRITE once by the ingestion script (scripts/ingest-rulings.ts); READ by every relief-finding and chat request.
- **filing_reliefs** — WRITE by /api/find-reliefs; READ by /api/calculate-tax and the results dashboard.
- **action_plans** — WRITE by /api/action-plan; READ by the results dashboard and (future) reminder service.

2.3.2. Normalized Database Schema



Key design decisions:

- The filing_reliefs table is a junction resolving the many-to-many relationship between filings and reliefs_master. Without it, we would either duplicate relief metadata per filing (denormalisation) or lose the per-filing AI reasoning and confidence score.

- rulings_vectors is deliberately denormalised from user input (chunk-level rather than ruling-level rows) because RAG retrieval works at the chunk level, not the ruling level. The ruling_code field provides a logical foreign key back to a conceptual ruling.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product

The following five core features constitute the MVP committed to UMHackathon 2026 preliminary-round demo. Each feature is testable end-to-end and has a clear pass/fail observable.

#	Features	Description
1	Smart Relief Finder with RAG Grounding	Eight-question lifestyle wizard produces a user profile. pgvector retrieves the top-5 most relevant LHDN ruling chunks. GLM reasons over them and returns a ranked list of eligible reliefs, each with a plain-language justification and an exact citation.
2	Deterministic Tax Estimator	Pure TypeScript engine applies 2026 tax bands (individual or SME as appropriate), relief caps, zakat rebate, and the RM 400 low-income rebate. Produces a full waterfall breakdown displayed as a chart and table.
3	Pre-Year-End Action Plan	GLM generates three ranked actions the user can still take before 31 December to reduce their YA 2026 tax, each with an estimated saving, a liquidity requirement, a deadline, and a ruling citation.
4	Ask SmarTax Chat (RAG-grounded)	Natural-language chat interface where users ask open-ended tax questions. Every response is grounded in retrieved LHDN rulings and ends with a clickable citation chip.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirements	Implementation
Scalability	Core domain modules (tax, glm, rag) must be independently testable and replaceable.	Each domain module exposes a typed interface. Zero imports from Next.js-specific packages into the domain modules. Vitest unit tests cover every public function in lib/tax/engine.ts.

Reliability	Core domain modules (tax, glm, rag) must be independently testable and replaceable.	Each domain module exposes a typed interface. Zero imports from Next.js-specific packages into the domain modules. Vitest unit tests cover every public function in lib/tax/engine.ts.
Maintainability	Core domain modules (tax, glm, rag) must be independently testable and replaceable.	Each domain module exposes a typed interface. Zero imports from Next.js-specific packages into the domain modules. Vitest unit tests cover every public function in lib/tax/engine.ts.
Token Latency	GLM calls return within 3 seconds at the 95th percentile.	5-second Async timeout on the API handler. Loading skeletons shown in UI while awaiting response. Response streaming is enabled for the chat endpoint to reduce time-to-first-token.
Cost Efficiency	Average token cost per user session must remain within the budget threshold.	Fixed-length system prompts; user input is truncated or chunked before injection; chat responses are cached for 24 hours using embedding similarity ≥ 0.92; scenario-simulator GLM calls are debounced to 500 ms of slider inactivity.
Security	User tax data is isolated per account; uploaded files are not exposed publicly.	Supabase Row-Level Security policies ensure users see only their own filings and chat history. Storage uploads use short-lived signed URLs. API keys for OpenAI and Supabase service-role are only present server-side (Vercel env vars), never shipped to the client.
Accessibility	MVP meets WCAG 2.1 AA for colour contrast and keyboard navigation.	shadcn/ui components are accessible by default. Focus rings preserved on all interactive elements. Colour palette passes AA contrast against both background variants.

3.3. Out of Scope / Future Enhancements

The following are identified as part of the SmarTax "grand vision" but not part of this UMHackathon 2026.

a. Year-round receipt wallet with photo capture and auto-categorisation into relief buckets.
b. Automatic re-ingestion of LHDN rulings when Budget updates are published.
c. Accountant/CPA co-pilot mode with multi-client dashboards.
d. Direct integration with LHDN e-Filing for one-click submission.
e. Native iOS and Android applications with offline-capable scanning.
f. Joint-vs-separate assessment optimiser for married couples.
g. Household / family mode allowing multiple filers in a single account.
h. Zakat-distribution guidance across approved collection bodies.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

After MVP, new features are released gradually using feature flags. A small percentage of users (e.g., 5%) will receive the update first, and system performance such as error rates and OpenAI usage cost will be monitored. If stable, the feature is progressively rolled out to all users. A/B testing is used to compare different prompt designs (e.g., relief suggestion prompts), with effectiveness measured by improvement in tax savings or user outcomes.

4.1.2. Emergency Rollback (ER) & Golden Release

Each deployment on Vercel generates a stable build. The "Golden Release" is the latest version that passes all checks, including linting, type validation, and tests. If issues occur after deployment—such as high error rates or abnormal API cost—the system can quickly revert to the last stable version using Vercel's rollback feature. A manual rollback guide is also maintained in the project documentation.

4.1.3. Priority Matrix:

Priority	Trigger Condition	Action
P0 (Critical)	Relief-finder error rate >5% OR every calculation returns an incorrect tax figure	Immediate emergency rollback to Golden Release. Incident postmortem within 24 hours.
P1 (High)	GLM cost per session> budget threshold + 30% OR p95 latency > 6 seconds sustained for 15 minutes	Rollback if the issue was introduced in the latest deployment. Otherwise, enable per-user rate limiting and aggressive cache TTLs.
P2 (Medium)	A single relief or chat answer is found to be incorrect but not systemic	Hotfix the ruling embedding or the specific prompt template within the next release cycle.
P3 (Low)	UI cosmetic issues, wording tweaks, non-critical copy errors	Track in GitHub issues. Batch into the next sprint.

4.2. Monitoring:

SmarTax uses four monitoring signals, each with an alert threshold wired to a developer Slack channel for the team and to email for the on-call rotation:

a. **Error rate** — Vercel logs surface 5xx responses. Alert when 5xx rate > 1% over 5 minute.
b. **API latency** — Request duration for OpenAI calls is logged. Alert when p95 latency exceeds 4 seconds.
c. **API cost** — Token usage per request is recorded, with a daily summary of cost per active user. Alert when daily cost exceeds 150% of the 7-day rolling average.
d. **Data-integrity checks** — A nightly script compares the sum of filing_reliefs amounts against the stored total_reliefs per filing; mismatches are flagged.

4.3. Assumptions:

- Users have stable internet connectivity (broadband or 4G+). Offline mode is not supported in the MVP.
- Users manually input accurate financial data (e.g., income, expenses). The system does not verify data authenticity.
- LHDN Public Rulings used for retrieval remain valid for the target assessment year. Any regulatory updates are monitored manually.
- OpenAI API is available with stable performance during the system's operation. Rate limits are considered but not expected to significantly impact usage.
- Users have basic proficiency in English or Bahasa Malaysia. Simplified guidance may be added in future versions.
- Supabase free tier limits are sufficient for the MVP usage, with a clear upgrade path for scaling.

4.4. External Dependencies:

Tools	Purpose	Risks
Open AI	Core reasoning engine for extraction, relief finding, action planning, and chat. Handles multi-step prompt inference and structured JSON output.	High — if the API is rate-limited or unavailable, core features fail. Mitigation: response caching, retry with exponential backoff, graceful degradation showing a "temporarily unavailable" state for AI features while the deterministic calculator remains usable.
OpenAI Embeddings API	Generate vector embeddings for semantic similarity search in chatbot	High — if the API is rate-limited or unavailable, the chatbot ends up doing word search in the Supabase. Mitigation:
Supabase	Database, auth, storage, vector search, and RPC endpoints.	Medium — free-tier service limits could be hit under unexpected load. Mitigation: monitoring of row counts and storage; paid-tier upgrade path documented. Row-level security enforced on every table.
Vercel	Application deployment and edge delivery	Low — stable provider with a long track record. Mitigation: alternative deploy targets (Netlify, Cloudflare Pages) could host the Next.js app with minor configuration changes.
LHDN Public Rulings (source content)	Authoritative text corpus for RAG.	Low — rulings are public and stable. Mitigation: local copies of every ruling PDF are committed to the repo so that ingestion is reproducible even if LHDN's website is temporarily down.
GitHub Actions	CI/CD pipeline.	Low — free minutes sufficient for hackathon scope.

5. Project Management & Team Contributions

5.1. Project Timeline

Days	Phase	Deliverables
Day 1–2	Foundation & Data	Repo scaffold with Next.js + Supabase + Z.AI integration; schema migrations executed; 15 LHDN Public Rulings ingested into pgvector; deployed skeleton reachable at a live URL; "hello GLM" API call verified end-to-end.
Day 3–4	Core Tax Engine	Deterministic TypeScript calculator for individual mode complete with Vitest unit tests; 22-question relief wizard UI; tax-breakdown waterfall display using Recharts.
Day 5–6	Open AI Integration	EA form details working end-to-end; relief-finder pipeline returns ranked list with citations; firewall cross-checks ruling codes; pre-year-end action plan endpoint live.
Day 7-8	SME + Freelancer Modes	SME-specific tax engine; P&L upload and GLM categorisation of deductible vs non-deductible; freelancer mode handling mixed employment + business income; landing-page mode switcher; zakat rebate optimisation; PWA manifest and service worker.
Day 9	Testing & Deck	Full E2E testing across all three personas; QA risk register finalised; pitch deck completed; backup demo video recorded.
Day 10	Rehearsal & Submission	Ten rehearsals of the 5-minute pitch; Q&A prep notes aligned with the risk register; final submission package including GitHub repo, live URL, and all three documentation deliverables (PRD, SAD, QATD).

5.2. Team Members Role

Role	Responsibility
Technical Lead / Full-Stack	Owns overall architecture, reviews all merges to main, builds the core tax engine and OpenAI service layer, presents the technical walkthrough.
Frontend Engineer	Builds React pages and shadcn/ui components; scenario simulator, and result dashboard; owns UI/UX polish.
AI / Prompt Engineer	Designs and iterates prompt templates; builds the RAG retrieval and firewall layers; authors the AI-output test suite in the QATD.
QA & Documentation Lead	Maintains the QATD; writes regression and integration tests; prepares demo script and backup video; coordinates pitch rehearsals.

5.3. Recommendations

Recommendations for post-MVP scaling, reliability, and cost optimisation:

- **Cache popular chat answers at the edge.**
  Questions such as "what is the lifestyle relief?" can be served from Vercel Edge Cache when their embedding similarity is ≥ 0.95, bypassing GLM entirely.

- **Use pgvector half-precision vectors.**
  Halves the storage footprint with negligible accuracy loss — relevant if the ruling corpus grows beyond 1,000 chunks.

- **Introduce Redis for session-level state.**
  If scenario simulator use expands, a Redis cache keyed by filing_id will reduce redundant Supabase round-trips.

- **Add Sentry or Axiom for error observability.**
  Vercel logs are adequate for the MVP but not for production debugging of serverless functions.

- **Introduce a nightly LHDN ruling update job.**
  A scheduled GitHub Action that re-ingests any rulings whose "last_modified" changed on hasil.gov.my would keep the corpus fresh without manual intervention.